Grafana Query Performance

Tracking System

Automated monitoring and optimization of PostgreSQL queries across 50+ dashboards

August 2026

SITUATION: Performance visibility gap in enterprise Grafana deployment

Scale Challenge

50+ dashboards
200+ panels
Hundreds of queries

No Visibility

No centralized monitoring
Reactive troubleshooting
User complaints first signal

Impact

Poor user experience
Resource waste
Decreased adoption

COMPLICATION: Manual monitoring doesn't scale

1. Cannot manually review pg_stat_statements for 50+ dashboards
2. No way to correlate PostgreSQL queries to specific Grafana panels
3. Performance issues discovered only after user impact
4. No historical data to identify trends or anomalies
5. Optimization efforts not prioritized by business impact

SOLUTION: Automated query performance tracking system

1. Data Collection

- Python script
- pg_stat_statements
- Scheduled snapshots
- Tag-based tracking

2. Storage

- Time-series DB
- Historical trends
- Performance metrics
- Aggregated stats

3. Visualization

- Health dashboard
- Real-time alerts
- Trend analysis
- Drill-down views

Proactive, data-driven query optimization across entire Grafana estate

TECHNICAL ARCHITECTURE

KEY FEATURES AND CAPABILITIES

Monitoring

✓ Real-time active query tracking (>30s)

✓ Slow query identification (configurable threshold)

✓ Historical performance trends

✓ Dashboard-level aggregation

Analysis

✓ Chatty × Slow ranking (calls × avg time)

✓ Variable vs. panel load time breakdown

✓ Estimated dashboard load time

✓ Standard deviation for consistency checks

Actionability

✓ Prioritized optimization backlog

✓ Direct links to slow dashboards

✓ Drill-down to panel level

✓ Tag-based query traceability

KEY METRICS AND PERFORMANCE INDICATORS

50+	200+	20+	100%
Dashboards Monitored	Panels Tracked	Slow Queries Identified	Query Visibility

Business Impact

- Proactive issue detection before user impact
- Data-driven optimization prioritization
- Reduced MTTR for performance incidents
- Improved user experience and platform adoption

IMPLEMENTATION HIGHLIGHTS

❌ **Query Identification**	✓ Implemented tag-based tracking (@grafana/uid/panel_id) embedded in query comments
❌ **Metric Aggregation**	✓ Regex extraction + GROUP BY to handle query variations and normalize data
❌ **Historical Analysis**	✓ Periodic snapshots with timestamps to track trends beyond cumulative stats
❌ **Performance Impact**	✓ Lightweight collector with indexed queries and off-peak scheduling
❌ **Scalability**	✓ Automated execution via Azure Functions with error handling and retry logic

TECHNOLOGY STACK

Backend	Python 3.8+	SQLAlchemy ORM	psycopg2 driver
Database	PostgreSQL 13+	pg_stat_statements extension	Time-series storage
Visualization	Grafana 9.0+	JSON dashboard definition	PostgreSQL datasource
Deployment	Azure Functions (optional)	Scheduled execution	Environment-based config

FUTURE ENHANCEMENTS

Automated Alerting

High

- Slack/email notifications for threshold breaches
- Anomaly detection using ML
- Auto-escalation workflows

Query Optimization AI

High

- Automatic index recommendations
- Query rewrite suggestions
- Materialized view candidates

Capacity Planning

Medium

- Predictive analytics for resource needs
- Cost projection models
- Growth trend forecasting

CI/CD Integration

Medium

- Performance regression testing
- Dashboard deployment gates
- Automated performance SLAs

Key Takeaways

1. Visibility is the foundation – you can't optimize what you don't measure
2. Proactive monitoring prevents user-impacting performance issues
3. Data-driven prioritization maximizes optimization ROI
4. Automation is essential for scaling beyond 5-10 dashboards
5. Historical trends reveal patterns that point-in-time snapshots miss

From reactive firefighting to proactive optimization